

Построение компилятора с помощью ANTLR

Описание атрибутивной грамматики средствами ANTLR 4

Что такое атрибутивная грамматика

- правила грамматики
- семантические действия
- для каждого символа грамматики могут задаваться атрибуты
- два способа передачи значений атрибутов:
- наследование
- синтезирование

Семантические действия (1)

Записываются на языке программирования и вставляются в правила используя `{ }`

```
decl: type ID ';' { System.out.println( "found a decl"); } ;  
type: 'int' | 'float' ;
```

Семантические действия (2)

Специальные действия `init` и `after`

```
decl  
@init { ... }  
@after { ... }  
: type ID ;
```

Семантические действия (3)

```
decl: type ID ';'
      {System.out.println("var"+SID.text+": "+Stype.text+");"}
  | t=ID id=ID ';'
      {System.out.println("var"+$id.text+": "+$t.text+");"}
  ;
```

Передача контекста в семантические действия

- `$ID` - подставляет объект элемента грамматики `ID`
- `$ID. text` - получаем значения атрибута `text` у элемента грамматики `ID`
- `$r::x` - получаем значения атрибута `x` для правила `г`. Это может быть сокращено до `$x`

Метки в правилах (1)

```
decl: t=ID id=ID ;
```

t и d-метки, позволяют в семантических действиях обратиться к конкретным элементам правила, если они имеют одинаковый тип

Метки в правилах (2)

```
array : '{' e1+=INT (',' e1+=INT)* '}';
```

e1 - метка, через которую можно обратиться к массиву разобранных INT

Встроенные атрибуты у терминалов (1)

- text (тип String) - фрагмент исходного текста, который соответствует терминалу. Пример: \$ID.text
- int (тип Integer) - если значение терминала может быть записано как число, то будет хранить это число
- type (тип Integer) - тип (индекс) терминала
- line (тип Integer) - номер строки, где находится терминал (нумерация с 1)

Встроенные атрибуты у терминалов (2)

- `pos` (тип `Integer`) - номер колонки (нумерация с 0)
- `index` (тип `Integer`) - индекс в потоке терминалов (нумерация с 0)
- `channel` (тип `Integer`) - индекс канала (по умолчанию 0)

Встроенные атрибуты у нетерминалов (1)

Каждому нетерминалу в грамматике соответствует правило(а).

- `text` (тип `String`) - фрагмент исходного текста, который соответствует нетерминалу.
- `start` (тип `Token`) - стартовый терминал правила
- `stop` (тип `Token`) - последний терминал правила
- `ctx` (тип `ParserRuleContext`) - контекст правила, через который можно обратиться к дополнительно объявленным атрибутам

Объявление дополнительных атрибутов (1)

```
rulename[args] returns [retvals] locals [localvars] : ... ;
```

- args
- retvals
- localvars

Объявление дополнительных атрибутов (2)

Если нужно инициализировать атрибуты из locals или returns можно использовать @init действие

```
row[String[] columns]
  returns [Map<String, String> values]
  locals [int col=0]
  @init {
    Svalues = new HashMap<String, String>();
  }
  : ...
;
```

Объявление дополнительных атрибутов (3)

```
add[int x] returns [int result]: '+= INT {Sresult = $x+ SINT.int;} ;
```

Для различных целевых языков задания в [] будут отличаться.

Java, C#, C++ - [int x = 32, float y]

Python, JavaScript - [x, y]

Доступ к объявленным атрибутам

```
block
  locals [
    List<String> symbols = new ArrayList<String>()
  ]
  : '{' decl* stat+ '}' {
    System.out.println("symbols=" + $symbols);
  }
  ;

decl: 'int' ID { $block: :symbols.add($ID.text); } ';'
;
```

Передача значений атрибутов в правила

```
decl: type declarator[$type.text] ';' ;  
declarator[String typeText]  
    : '*' ID { System.out.println("var " + $ID.text + $typeText); }  
    | ID { System.out.println("var " + $ID.text + $typeText) ; }  
    ;
```

Возвращение значений из правила

```
field
    : d=decl ';' {
        System.out.println("type "+$d. type+", vars="+$d. vars);
    }
;
decl returns [String type, List vars]
    : t=type ids+=ID ('' ids+=ID)* { Stype = St. text; Svars = $ids; }
;
```

Семантические предикаты (1)

```
stat: decl | expr ;  
decl: ID ID ;  
expr:  
    {istype()}? ID '(' expr ')' // ctor-style typecast  
    | {isfunc()}? ID '(' expr ')' // function call  
    ;
```

Семантические предикаты (2)

```
stat: decl | {java5}? expr ;
```

java5 - логическая переменная

getCurrentToken() - возвращает текущий токен, можно использовать в коде семантических предикатов

Семантические предикаты (3)

```
vec5
locals [int i=1]
      : ( { $\$i \leq 5$ }? INT { $S i++;$ } )* / match 5 INTs
      ;
```

Опции для элементов правил (1)

Общий вид : I<name=value>

Используется assoc и fail

```
expr: expr '^'<assoc=right> expr  
      ;
```

Опции для элементов правил (2)

```
ints[int max]
  locals [int i=1]
  : INT ( ' ,' {Si++;} {$i<=$max}?<fail={"exceeded max " + $max}> INT )*
```

```
{...}?<fail={doSomethingAndReturnAString()}>
```

Обработка исключительных ситуаций в правилах (1)

```
r: ...  
    ;  
    catch [RecognitionException e] { ... }
```

ИЛИ

```
r: ...  
    catch [FailedPredicateException fpe] { ... }  
    catch [RecognitionException e] { ... }
```

Обработка исключительных ситуаций в правилах (2)

```
r: ...  
    ;  
    // catch blocks go first  
    finally { System.out.println("exit rule r"); }
```

Действия на уровне грамматики

```
grammar Count;  
@header {  
    package foo;  
}  
@members {  
    int count = 0;  
}
```

Visitor v s Listener

- где писать логику семантических правил и генерации промежуточного кода
- Visitor == DOM, Listener == SAX